

Descriptive Statistics in Finance

Using Python for Derivatives & Risk Management

ECON F354 / FIN F311 BITS Pilani, Pilani Campus

Module 1: Describing the Past

with brief introductions to Describing the Present and What Bears on the Future

1. Introduction

This document accompanies the Jupyter notebook `descriptive_stats_drm.ipynb` and provides the theoretical foundation for each statistical measure covered. Every section follows the same pattern:

1. **Definition** — What the statistic is, with formula
2. **Interpretation** — What it tells you about the data
3. **Explanation** — Why it matters for risk management
4. **Python Code** — How to compute it

1.1. How to Use This Material

- Read the definition and interpretation first. Run the corresponding Python cell in the notebook to see the statistic computed on real NSE data (NIFTY 50 and Reliance Industries, 2020–present). Compare the numbers across assets and against the theoretical benchmarks (e.g., normal distribution reference values). Answer the discussion questions at the end of each major section.

1.2. Prerequisites

Students should be familiar with:

- Time value of money (present value, future value, discounting)
- Basic financial markets (equities, indices, risk-free rate)
- Types of derivatives (forwards, futures, options, swaps)
- Basic Python (variables, functions, lists, importing libraries)

No prior statistics or programming experience beyond these is assumed.

1.3. Why Python?

All computations in this module are done in Python — the dominant language in quantitative finance. Every derivative pricing desk, risk management team, and algorithmic trading firm uses Python (or a similar language) for data analysis. The code you see in the notebook is production-style code: the exact same functions (`numpy`, `scipy`, `pandas`) are used at banks, hedge funds, and asset managers worldwide.

2. Part A: Describing the Past

“The past is fully measurable. These statistics tell you what the data actually looked like.”

2.1. A1. Why Returns, Not Prices?

2.1.1. Definition

Financial statistics are computed on **log returns**, not raw prices:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1})$$

2.1.2. Interpretation

Prices are **non-stationary** — their mean drifts over time and their variance grows with the price level. A ₹100 stock moving ₹5 is 5%; a ₹1,000 stock moving ₹5 is 0.5%. Raw price changes are not comparable across stocks or across time.

Log returns solve this:

- They oscillate around zero regardless of the price level
- They are **time-additive**: $r_{t \rightarrow t+4} = r_t + r_{t+1} + r_{t+2} + r_{t+3} + r_{t+4}$
- They are directly comparable across assets
- Unlike simple returns, they are symmetric: $\ln\left(\frac{110}{100}\right) = 0.0953$ while $\ln\left(\frac{100}{110}\right) = -0.0953$

2.1.3. Explanation (Risk Management Context)

Every statistic that follows — mean, standard deviation, skewness, VaR — assumes the data is stationary. Running them on prices would produce numbers that are meaningless. The notebook’s visualization confirms this: prices drift with a trend, but returns hover around a stable mean near zero.

2.1.4. Python Code

```
prices = raw['Close'].copy()
log_returns = np.log(prices / prices.shift(1)).dropna()
simple_returns = prices.pct_change().dropna()
```

2.2. Location (Central Tendency)

2.2.1. A2. Arithmetic Mean

2.2.1.1. Definition

$$\vec{r} = \frac{1}{n} \sum_{t=1}^n r_t$$

Annualised: $\vec{r}_{\text{annual}} = \vec{r}_{\text{daily}} \times 252$ (NSE has 252 trading days per year).

2.2.1.2. Interpretation

The mean tells you the **average daily return** over the sample period. It is the most natural measure of “central tendency.” However, it is sensitive to outliers — a single +10% or −13% day can noticeably shift the mean.

2.2.1.3. Explanation (Risk Management)

For a risk manager, the mean is the starting point for expected return calculations. But never use it alone. Always compare with the median. A large gap between mean and median signals extreme days. Always annualise when comparing across time frames.

2.2.1.4. Python Code

```
mean_daily = log_returns.mean()  
mean_annual = mean_daily * 252
```

2.2.2. A3. Median

2.2.2.1. Definition

The middle value when all observations are sorted. The 50th percentile of the data.

2.2.2.2. Interpretation

The median is the **robust** measure of central tendency — unaffected by how extreme the outliers are. A −13% crash and a −5% bad day have the same effect on the median (both are “below-median”). If the mean is lower than the median, it signals negative skew.

2.2.2.3. Explanation (Risk Management)

The median is often a better estimate of “what a normal day looks like” than the mean. Many practitioners prefer robust estimators precisely because the mean is so easily influenced by crisis days that may not repeat.

2.2.2.4. Python Code

```
median_daily = log_returns.median()  
gap = mean_daily - median_daily # positive gap → mean pulled up by outliers
```

2.2.3. A4. CAGR (Compound Annual Growth Rate)

2.2.3.1. Definition

$$\text{CAGR} = \left(\frac{P_T}{P_0} \right)^{\frac{1}{n}} - 1$$

where n is the number of years. Equivalently: $\text{CAGR} = \exp(\vec{r}_{\text{annual}}) - 1$.

2.2.3.2. Interpretation

The CAGR tells you what you **actually** earned per year, accounting for compounding. A +10% day followed by a −10% day leaves you at −1% overall — the arithmetic mean is 0%, but the geometric mean is approximately −0.5%.

2.2.3.3. Explanation (Risk Management)

CAGR is the most honest measure of long-run returns. When evaluating a fund or strategy, always ask for CAGR, not the average annual return. The difference is the **volatility drag**.

2.2.3.4. Python Code

```
n_years = len(prices) / 252
cagr = (prices.iloc[-1] / prices.iloc[0]) ** (1 / n_years) - 1
geo_daily = (1 + simple_returns).prod() ** (1 / len(simple_returns)) - 1
```

2.2.4. A5. Volatility Drag

2.2.4.1. Definition

$$\text{Drag} = \vec{r}_{\text{annual}} - \text{CAGR}$$

Approximation (from Jensen's inequality): $\text{Drag} \simeq \frac{\sigma^2}{2}$, where σ^2 is the variance of log returns (annualised).

2.2.4.2. Interpretation

The drag tells you how much return you **lose** to volatility. A ₹100 investment that goes up 50% then down 50% ends at ₹75, not ₹100. The arithmetic average return is 0%, but the compound return is -13.4%. Higher volatility reduces your compound return even if the average return stays the same.

2.2.4.3. Explanation (Risk Management)

Volatility drag is why risk management matters for long-term returns. Reducing portfolio volatility through diversification directly improves compound return. A risk manager who only looks at Sharpe ratios without understanding volatility drag is missing half the story.

2.2.4.4. Python Code

```
daily_var = log_returns.var()
approx_drag = 0.5 * daily_var * 252
actual_drag = mean_annual - cagr
```

2.3. Scale (Dispersion)

2.3.1. A6. Standard Deviation & Variance

2.3.1.1. Definition

$$\sigma^2 = \frac{1}{n-1} \sum_{t=1}^n (r_t - \bar{r})^2$$
$$\sigma = \sqrt{\sigma^2}$$

Annualised: $\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$.

2.3.1.2. Interpretation

Standard deviation (σ) is the most widely used measure of risk. In a normal distribution: 68% of days fall within $\pm 1\sigma$, 95% within $\pm 2\sigma$, 99.7% within $\pm 3\sigma$. But financial returns are **not** normal, so these percentages do not hold exactly.

The $\sqrt{252}$ annualisation rule assumes daily returns are independent. They are not perfectly independent, but the convention is universal.

2.3.1.3. Explanation (Risk Management)

Volatility is the primary input to position sizing, margin calculations, and risk limits. A 1% daily vol means roughly 16% annual vol. The limitation: standard deviation treats upside and downside identically. A stock that gaps up 10% contributes as much to σ as one that gaps down 10%.

2.3.1.4. Python Code

```
daily_std = log_returns.std()
annual_vol = daily_std * np.sqrt(252)
```

2.3.2. A7. Mean Absolute Deviation (MAD)

2.3.2.1. Definition

$$\text{MAD} = \frac{1}{n} \sum_{t=1}^n |r_t - \bar{r}|$$

2.3.2.2. Interpretation

MAD is less sensitive to outliers than standard deviation. For a normal distribution, $\sigma \simeq 1.25 \times \text{MAD}$. If $\frac{\sigma}{\text{MAD}}$ is much larger than 1.25, it signals heavy tails – extreme observations are inflating the standard deviation relative to what a “typical” deviation looks like.

2.3.2.3. Explanation (Risk Management)

MAD is more stable over time than σ . If you are setting risk limits that should not jump around every time there is a market event, MAD-based limits may be preferable.

2.3.2.4. Python Code

```
mad = (log_returns - log_returns.mean()).abs().mean()
sigma_over_mad = daily_std / mad
```

2.3.3. A8. Interquartile Range (IQR)

2.3.3.1. Definition

$$\text{IQR} = Q_3 - Q_1$$

where Q_1 is the 25th percentile and Q_3 is the 75th percentile.

2.3.3.2. Interpretation

The IQR tells you the spread of the “typical” half of the distribution, completely ignoring the most extreme 25% on each side. It is **extremely robust** – even if the worst day were –50% instead of –13%, the IQR would not change at all. Outliers are flagged when an observation lies more than $1.5 \times \text{IQR}$ beyond Q_1 or Q_3 .

2.3.3.3. Explanation (Risk Management)

If σ suggests high risk but the IQR is modest, most of the “risk” is coming from a few extreme days — and those may or may not repeat.

2.3.3.4. Python Code

```
Q1, Q3 = log_returns.quantile(0.25), log_returns.quantile(0.75)
IQR = Q3 - Q1
outlier_low, outlier_high = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
```

2.3.4. A9. Coefficient of Variation (CV)

2.3.4.1. Definition

$$CV = \frac{\sigma}{|\bar{r}|}$$

Risk per unit of return.

2.3.4.2. Interpretation

An asset with $\sigma = 20\%$ and mean return = 10% has $CV = 2.0$. An asset with $\sigma = 15\%$ and mean return = 3% has $CV = 5.0$ — it is **riskier** per unit of return despite lower absolute volatility. Lower CV indicates a more “efficient” risk-return trade-off.

2.3.4.3. Explanation (Risk Management)

CV enables risk efficiency comparisons across asset classes. But it breaks down when the mean is near zero ($CV \rightarrow \infty$) and treats upside and downside symmetrically.

2.3.4.4. Python Code

```
cv = daily_std / abs(mean_daily)
```

2.4. Distribution Shape

2.4.1. A10. Skewness

2.4.1.1. Definition

$$S = \frac{1}{n} \sum_{t=1}^n \left(\frac{r_t - \bar{r}}{\sigma} \right)^3$$

Normal distribution: $S = 0$.

2.4.1.2. Interpretation

- $S > 0$: Right-skewed — large positive returns more common (commodity rallies, short-squeeze events)
- $S < 0$: Left-skewed — large negative returns more common (“crash risk”)
- $S = 0$: Symmetric

Equity index returns are almost always negatively skewed. Markets take the stairs up and the elevator down.

2.4.1.3. Explanation (Risk Management)

Negative skewness is a red flag. It means VaR models assuming symmetry will underestimate downside risk. Black-Scholes assumes lognormal returns (slight positive skew); real equity returns have the **opposite sign**. A negatively skewed portfolio needs more capital buffer than a symmetric one with the same volatility.

2.4.1.4. Python Code

```
from scipy import stats
skew = log_returns.apply(stats.skew)
```

2.4.2. A11. Kurtosis (Excess)

2.4.2.1. Definition

$$K = \frac{1}{n} \sum_{t=1}^n \left(\frac{r_t - \bar{r}}{\sigma} \right)^4$$

Normal distribution has $K = 3$ (raw). Excess kurtosis = $K - 3$ (zero for normal).

2.4.2.2. Interpretation

- $K_{\text{excess}} > 0$: Leptokurtic (fat tails) — hallmark of financial returns
- $K_{\text{excess}} = 0$: Mesokurtic (normal tails)
- $K_{\text{excess}} < 0$: Platykurtic (thin tails) — rare in finance

Typical equity returns have excess kurtosis between 2 and 10+. A 5-sigma event should occur once every 7,000 years under normality; in equity markets, they happen every few years.

2.4.2.3. Explanation (Risk Management)

Kurtosis is arguably the single most important distributional fact for risk managers. It means VaR at 99% confidence is much larger than normal models predict. Portfolio insurance (put options) is more valuable than Black-Scholes suggests. Excess kurtosis is why we need GARCH, EVT, and Cornish-Fisher VaR — topics covered in Module 3.

2.4.2.4. Python Code

```
excess_kurt = log_returns.apply(stats.kurtosis) # Fisher = excess
raw_kurt = log_returns.apply(lambda x: stats.kurtosis(x, fisher=False))
```

2.5. Normality Tests

2.5.1. A12. Jarque-Bera Test

2.5.1.1. Definition

$$JB = \frac{n}{6} \left(S^2 + \frac{(K - 3)^2}{4} \right)$$

Under H_0 (normality), JB follows $\chi^2(2)$. Reject if $p < 0.05$.

2.5.1.2. Interpretation

The JB test jointly tests whether skewness = 0 and kurtosis = 3. For daily equity returns, the test statistic is typically in the thousands — overwhelming rejection of normality (the $\chi^2(2)$ critical value at 5% is only 5.99). Important caveat: JB can fail to reject normality for small samples ($n < 100$).

2.5.1.3. Explanation (Risk Management)

The JB test is the first formal evidence that “returns are normal” is a bad assumption. Every model that assumes normality — Black-Scholes, simple parametric VaR, mean-variance optimisation — is now suspect.

2.5.1.4. Python Code

```
jb_stat, jb_p = stats.jarque_bera(log_returns['NIFTY 50'].dropna())
# Critical  $\chi^2(2)$  at 5% = 5.99
```

2.5.2. A13. Shapiro-Wilk Test

2.5.2.1. Definition

The Shapiro-Wilk test compares ordered sample values with expected order statistics of a normal distribution. W ranges from 0 to 1; $W = 1$ indicates perfect normality.

2.5.2.2. Interpretation

Unlike JB, Shapiro-Wilk is a general-purpose normality test — it has good power against a wide range of alternatives. It is most powerful for smaller samples ($n < 2000$). For larger datasets, a random sample is drawn.

2.5.2.3. Explanation (Risk Management)

Running multiple normality tests is itself a risk management discipline. No single test catches every departure from normality. If JB, Shapiro-Wilk, and Anderson-Darling **all** reject, you can be confident that normality-based models will misprice risk.

2.5.2.4. Python Code

```
sample = returns.sample(min(5000, len(returns)), random_state=42)
sw_stat, sw_p = stats.shapiro(sample)
```

2.5.3. A14. Anderson-Darling Test

2.5.3.1. Definition

A modification of the Kolmogorov-Smirnov test giving more weight to the tails of the distribution — exactly where financial returns deviate most from normality. If the test statistic exceeds the critical value at the chosen significance level, normality is rejected.

2.5.3.2. Interpretation

The AD test is the most relevant normality test for risk management because it is **tail-sensitive**. A distribution could have skewness ≈ 0 and kurtosis ≈ 3 overall but still fail AD because the extreme tails deviate from normality. Think of the three tests as complementary: JB checks moments, Shapiro-Wilk checks overall shape, and Anderson-Darling checks the tails.

2.5.3.3. Explanation (Risk Management)

Tail behaviour drives VaR, Expected Shortfall, and stress test outcomes. AD is the one test most likely to stay rejected even after outlier treatment — because fat tails are intrinsic to equity returns, not an artefact of a few extreme days.

2.5.3.4. Python Code

```
ad = stats.anderson(returns.dropna(), dist='norm')
rejected_5pct = ad.statistic > ad.critical_values[2]
```

2.5.4. A15. Quantile Profile (Percentiles)

2.5.4.1. Definition

q_p is the value below which proportion p of the observations fall.

Percentile	Risk Management Use
------------	---------------------

1%	Historical VaR at 99% confidence
5%	Historical VaR at 95% confidence
50%	Median (central tendency)
95%	Best 5% of days
99%	Best 1% of days

2.5.4.2. Interpretation

Quantiles answer directly what a “bad day” looks like. The 1st percentile tells you: **on the worst 1% of days, how bad was it?** Standard deviation cannot answer this. The gap between actual quantiles and normal-predicted quantiles is the error from assuming normality for VaR.

2.5.4.3. Explanation (Risk Management)

Quantile-based thinking is central to modern risk management. The Basel Committee requires banks to hold capital against a 99.9% VaR. Expected Shortfall (CVaR) requires knowing the entire tail shape, not just the 99th percentile.

2.5.4.4. Python Code

```
quantiles = [0.01, 0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99]
q_actual = log_returns.quantile(quantiles)
# Compare with normal:
q_normal = stats.norm.ppf(quantiles, loc=mu, scale=sigma)
```

2.6. Path-Dependent Risk

2.6.1. A16. Maximum Drawdown (MDD)

2.6.1.1. Definition

A drawdown is the decline from a historical peak:

$$D_t = \frac{P_t - \max_{s \leq t} P_s}{\max_{s \leq t} P_s}$$

$$\text{MDD} = \min_{t \in [0, T]} D_t$$

2.6.1.2. Interpretation

MDD answers: “**What was the worst possible time to buy and the worst possible time to sell?**”

It is the risk measure investors actually **feel**. A 25% annual volatility tells you nothing about whether the worst decline was 20% or 60%. For NIFTY 50 (2020–2026), the worst drawdown occurred during the COVID crash of March 2020 (40% peak-to-trough).

2.6.1.3. Explanation (Risk Management)

MDD is what fund managers and clients actually discuss. Many mandates include explicit MDD limits. MDD also governs leverage: $2\times$ leverage roughly doubles the MDD. A 40% MDD with $2\times$ leverage means an 80% drawdown — effectively ruin for most strategies.

2.6.1.4. Python Code

```
running_max = series.cummax()
drawdown = (series - running_max) / running_max
mdd = drawdown.min()
```

2.6.2. A17. Drawdown Duration & Time Under Water

2.6.2.1. Definition

Drawdown duration is the number of days from a peak until the price recovers to that peak. **Time under water** is the percentage of total trading days the price is below a previous all-time high.

2.6.2.2. Interpretation

MDD tells you **how far** you fell; duration tells you **how long** you stayed down. A 20% drawdown lasting two years can be more psychologically damaging than a 40% crash that recovers in six months. NIFTY 50 typically spends 90%+ of trading days below previous all-time highs.

2.6.2.3. Explanation (Risk Management)

Drawdown duration determines how long capital is tied up. Two years underwater means two years of being unable to withdraw without locking in a loss. Pension funds and endowments care about duration as much as depth.

2.6.2.4. Python Code

```
in_dd = drawdown < -0.0001
time_under_pct = in_dd.sum() / len(drawdown)
```

2.6.3. A18. Ulcer Index

2.6.3.1. Definition

$$\text{UI} = \sqrt{\frac{1}{n} \sum_{t=1}^n D_t^2}$$

where D_t is the drawdown at time t . Like standard deviation, it is a root-mean-square measure — but of drawdowns, not returns.

2.6.3.2. Interpretation

The Ulcer Index only penalises drawdowns — upside volatility contributes zero. For a long-only investor, UI may be a better risk measure than σ because it captures what they actually care about: being underwater. UI is always lower than annualised volatility.

2.6.3.3. Explanation (Risk Management)

Developed by Peter Martin (1987), the UI aligns with how clients experience risk. It is used in the

Martin Ratio: UI Performance = $\frac{\text{CAGR} - R_f}{\text{UI}}$, an alternative to the Sharpe ratio.

2.6.3.4. Python Code

```
drawdown = (series - series.cummax()) / series.cummax()
ulcer_index = np.sqrt((drawdown ** 2).mean())
```

2.7. Risk-Adjusted Ratios

2.7.1. A19. Sharpe Ratio

2.7.1.1. Definition

$$\text{Sharpe} = \frac{R_p - R_f}{\sigma_p}$$

where R_p is the annualised return, R_f is the risk-free rate (use 6.5% for India), and σ_p is the annualised standard deviation.

2.7.1.2. Interpretation

Sharpe answers: **for each unit of risk taken, how much extra return did you earn over a risk-free bond?** Sharpe > 1 is good, > 2 is excellent, and negative means the risk-free asset outperformed. Limitation: it penalises upside volatility — a fund with consistent +3% days and -1% days may have a mediocre Sharpe despite being attractive.

2.7.1.3. Explanation (Risk Management)

Sharpe is the starting point, not the end. Always complement with Sortino (downside-only) and Calmar (drawdown-based). Indian equity Sharpe ratios are structurally lower than US/EU because the risk-free rate (6.5% G-Sec proxy) is higher — the bar for “excess return” is higher.

2.7.1.4. Python Code

```
Rf_daily = 0.065 / 252
excess = log_returns - Rf_daily
sharpe = excess.mean() / excess.std() * np.sqrt(252)
```

2.7.2. A20. Sortino Ratio

2.7.2.1. Definition

$$\text{Sortino} = \frac{R_p - R_f}{\sigma_{\text{down}}}$$

$$\sigma_{\text{down}} = \sqrt{\frac{1}{n} \sum_{r_t < \text{target}} (r_t - \text{target})^2}$$

2.7.2.2. Interpretation

Sortino fixes Sharpe’s main weakness: only returns below the target (typically 0 or R_f) contribute to risk. For assets with positive skew, Sortino > Sharpe. For equity indices (negative skew), Sortino is typically slightly lower than Sharpe. A higher Sortino relative to Sharpe suggests the “bad days” are less volatile than overall volatility suggests.

2.7.2.3. Explanation (Risk Management)

For long-only equity portfolios, Sortino is arguably more relevant than Sharpe. It is also more appropriate for strategies with asymmetric payoffs: options selling, trend-following, and any strategy with embedded optionality.

2.7.2.4. Python Code

```
below = excess[excess < 0]
downside_std = np.sqrt((below ** 2).mean())
sortino = excess.mean() / downside_std * np.sqrt(252)
```

2.7.3. A21. Calmar Ratio

2.7.3.1. Definition

$$\text{Calmar} = \frac{\text{CAGR}}{|\text{MDD}|}$$

2.7.3.2. Interpretation

Calmar answers: **was the return worth the worst-case pain?** A strategy with 15% CAGR and 30% MDD has Calmar = 0.50. A strategy with 10% CAGR and 10% MDD has Calmar = 1.0 — better return

per unit of worst-case loss. Unlike Sharpe (which can be gamed), Calmar cannot be manipulated — the MDD is what it is.

2.7.3.3. Explanation (Risk Management)

Calmar naturally penalises strategies that “blow up.” A fund with years of steady 15% returns and a single -60% drawdown will have a poor Calmar, correctly reflecting hidden tail risk. For setting position limits: if the target Calmar is 0.5 and historical MDD is 40%, you need CAGR $\geq 20\%$.

2.7.3.4. Python Code

```
calmar = cagr / abs(mdd)
```

2.7.4. A22. Omega Ratio

2.7.4.1. Definition

$$\Omega(L) = \frac{\sum_t \max(r_t - L, 0)}{\sum_t \max(L - r_t, 0)}$$

where L is a threshold (typically 0 or R_f). Numerator = total gains above threshold; denominator = total losses below it. $\Omega > 1$ means gains outweigh losses.

2.7.4.2. Interpretation

Omega’s key advantage: it does **not** assume normality. It simply sums all good outcomes and all bad outcomes and compares them. Valid for any return distribution, no matter how skewed or fat-tailed. An Omega of 1.5 means that for every ₹1 of losses, you earned ₹1.50 of gains.

2.7.4.3. Explanation (Risk Management)

Omega is gaining traction because it avoids the theoretical problems of moment-based ratios. It works for non-normal distributions, strategies with embedded options, and any time horizon. The downside: it lacks the intuitive scale of Sharpe (yet).

2.7.4.4. Python Code

```
gains = returns[returns > threshold].sum()
losses = abs(returns[returns < threshold].sum())
omega = gains / losses
```

3. Part B: Describing the Present

“The present is estimable. These tools tell you where the market is right now relative to its own recent history.”

3.1. B1. Beyond Close-to-Close Volatility

The standard deviation in Part A uses only closing prices. But each day produces four prices — Open, High, Low, Close (OHLC). Using all four gives a more efficient volatility estimate.

Estimator	Uses	Handles overnight gaps?
Close-to-Close	Close only	No
Parkinson	High, Low	No
Garman-Klass	Open, High, Low, Close	No
Rogers-Satchell	Open, High, Low, Close	No (drift-independent)
Yang-Zhang	Open, High, Low, Close	Yes

Yang-Zhang is the recommended estimator for Indian equities because it handles overnight gaps (the NSE closes at 3:30 PM; global news arrives before the 9:15 AM open). These estimators are typically 4–8× more efficient than close-to-close.

The Parkinson estimator (simplest of the OHLC family):

$$\sigma_{\text{Parkinson}} = \sqrt{\frac{1}{4 \ln 2} \cdot \frac{1}{n} \sum_{t=1}^n \left(\ln \left(\frac{H_t}{L_t} \right) \right)^2}$$

```
parkinson_vol = np.sqrt(1 / (4 * np.log(2)) *
    ((np.log(ohlc['High'] / ohlc['Low'])) ** 2).mean())
```

3.2. B2. EWMA Volatility

A rolling 21-day standard deviation treats every day equally. EWMA (Exponentially Weighted Moving Average) gives more weight to recent observations:

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2$$

where $\lambda \simeq 0.94$ (RiskMetrics convention). EWMA responds faster to volatility regime changes — rising sharply during crises and decaying gradually afterward.

```
lmbda = 0.94
ewma_var = np.zeros(len(returns))
ewma_var[0] = returns.iloc[0] ** 2
for t in range(1, len(ewma_var)):
    ewma_var[t] = lmbda * ewma_var[t-1] + (1 - lmbda) * returns.iloc[t] ** 2
ewma_vol = np.sqrt(ewma_var * 252)
```

3.3. B3. Regime & Position Descriptors

These tell you where today's observation sits relative to the recent past:

- **Z-score:** $\frac{P_t - \text{MA}_n}{\sigma_n}$ — how many standard deviations from the moving average
- **Bollinger %B:** $\frac{P_t - \text{Lower Band}}{\text{Upper Band} - \text{Lower Band}}$
- **RSI (Relative Strength Index):** speed of price changes on a 0–100 scale (> 70 = overbought, < 30 = oversold)
- **ATR (Average True Range):** volatility measure using the full day's range

```
delta = prices.diff()
gain = delta.clip(lower=0).rolling(14).mean()
```

```
loss = (-delta.clip(upper=0)).rolling(14).mean()
rsi = 100 - (100 / (1 + gain / loss))
```

3.4. B4. Relative Statistics: Beta

Beta (β) measures how much a stock moves when the index moves:

$$r_{\text{stock}} = \alpha + \beta \cdot r_{\text{NIFTY}} + \varepsilon$$

A β of 1.2 means a 1% NIFTY move implies a 1.2% stock move (on average). Rolling correlation reveals that correlations rise sharply during crashes — exactly when diversification is supposed to help.

```
rolling_beta = (
    stock_returns.rolling(252).cov(index_returns) /
    index_returns.rolling(252).var()
)
```

4. Part C: What Bears on the Future

“The future is only partly forecastable. But you can say useful things about risk even when you cannot predict direction.”

This is a brief preview. A full treatment is covered in Module 3.

4.1. C1. Volatility is Genuinely Forecastable

This is one of the most robust findings in empirical finance. Squared returns show significant positive autocorrelation for weeks or months — even though raw returns show almost none. This phenomenon is **volatility clustering**: volatile days tend to be followed by volatile days.

If these diagnostics confirm clustering (and they almost always do for equities), the workhorse model is **GARCH(1,1)**:

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2$$

The persistence parameter $\alpha + \beta$ (typically 0.95 for equities) tells you how slowly volatility shocks decay. A value near 1 means a shock today affects volatility forecasts for months.

4.2. C2. Returns Are Largely Not Predictable

For liquid large-cap stocks, there is very little exploitable serial structure:

- **ACF of raw returns** — typically zero at all lags
- **Hurst exponent** — near 0.5 (random walk)
- **ADF test** — confirms the log price has a unit root (random walk)

This is why we work with returns in the first place — prices are non-stationary (unit root), and returns are the transformation that makes the series stationary and amenable to statistical analysis.

4.3. C3. Tail Risk is Estimable Even When Direction Isn't

You cannot predict whether tomorrow is up or down. But you **can** say useful things about how bad a bad day could be:

- **Historical VaR**: the 1st or 5th percentile of historical returns
- **Cornish-Fisher VaR**: adjusts the normal VaR for the skewness and kurtosis measured in Part A
- **CVaR / Expected Shortfall**: the average loss **beyond** VaR
- **Extreme Value Theory (EVT)**: fits a separate distribution to tail observations only

These topics — GARCH modelling, VaR estimation, and EVT — form the core of Module 3. The statistics learned in Part A (skewness, kurtosis, quantiles, normality tests) are the essential foundation.

5. Summary

You have now studied **22 descriptive statistics** on real NSE data, organised across the three horizons of risk analysis:

Horizon	Statistics Covered
Past (fully measurable)	Mean, median, CAGR, volatility drag, σ , MAD, IQR, CV, skewness, kurtosis, JB test, Shapiro-Wilk, Anderson-Darling, quantile profile, MDD, drawdown duration, Ulcer Index, Sharpe, Sortino, Calmar, Omega
Present (estimable)	Parkinson vol, EWMA, RSI, rolling beta
Future (partly forecastable)	GARCH(1,1), VaR, CVaR, EVT (preview)

5.1. Key Takeaways

- **Run everything on returns, never on prices.** Prices are non-stationary. **Returns are not normal.** All three normality tests reject H_0 at $p < 0.0001$. This has profound implications for VaR, option pricing, and portfolio optimisation. **Volatility drag is real.** Higher volatility reduces compound returns even if average returns are unchanged (by approximately $\frac{\sigma^2}{2}$ per year). **The sequence of returns matters.** MDD and the Ulcer Index capture path-dependent risk that σ cannot. **No single statistic tells the full story.** Report mean, median, σ , IQR, skewness, kurtosis, and MDD together for a complete picture. **These are backward-looking.** They describe your sample, not the population. Structural breaks (COVID, policy changes) mean historical averages may not describe any single future year. Reserve a holdout period for validation.

5.2. What Comes Next

Module 2 – Describing the Present: Full treatment of OHLC volatility estimators, EWMA, regime descriptors, and relative statistics.

Module 3 – What Bears on the Future: GARCH family models, Value at Risk, Expected Shortfall, and Extreme Value Theory.

Module 4 – Options & Derivatives: Black-Scholes, implied volatility, the Greeks, and option payoff diagrams.